

Sistema de análisis digital de oscilaciones gestuales en pacientes con párkinson mediante inteligencia artificial

Esp. Ing. Alex Damián Barria Cárdenas

Maestría en Inteligencia Artificial

Director: PhD. Ing. Fernando Bernuy (Universidad Nacional de Chile)

Jurados:

Jurado 1 (pertenencia)
Jurado 2 (pertenencia)
Jurado 3 (pertenencia)

Ciudad de Buenos Aires, mayo de 2026

Resumen

La enfermedad de Parkinson altera el control motor y suele evaluarse en consulta mediante la observación de ejercicios estandarizados, lo que puede introducir subjetividad y dificultar el seguimiento fino de la evolución. En este trabajo se desarrolló un sistema de software que analiza grabaciones de video de personas que realizan esos ejercicios y produce medidas cuantitativas del movimiento para complementar la valoración clínica con información objetiva y trazable.

Se aplicaron contenidos de la maestría en inteligencia artificial, en particular visión por computadora, aprendizaje automático con modelos preentrenados, ingeniería de software y operaciones de aprendizaje automático.

Agradecimientos

Esta sección es para agradecimientos personales y es totalmente **OPCIONAL**.

Índice general

Resumen	I
1. Introducción general	1
1.1. Introducción general al tema	1
1.2. Contexto y motivación	2
1.3. Estado del arte	3
1.4. Objetivos del trabajo	4
1.4.1. Objetivo general	4
1.4.2. Objetivos específicos	4
2. Introducción específica	7
2.1. Frameworks y bibliotecas	7
2.2. Datasets y datos	8
2.3. Modelos preentrenados	10
2.4. Infraestructura	12
3. Diseño e implementación	13
3.1. Análisis del software	13
4. Ensayos y resultados	15
4.1. Pruebas funcionales del hardware	15
5. Conclusiones	17
5.1. Conclusiones generales	17
5.2. Próximos pasos	17
Bibliografía	19

Índice de figuras

1.1. Ejercicio de finger tapping utilizado en la evaluación clínica de movimientos finos de la mano.	2
2.1. Ejemplo ilustrativo de los puntos de referencia entregados por el modelo <code>HandLandmarker</code> de <code>MediaPipe</code> sobre la imagen de una mano.	10

Índice de tablas

1.1. Enfoques frecuentes para cuantificar el movimiento en Parkinson y contextos afines.	4
2.1. Bibliotecas y herramientas de terceros utilizadas en el desarrollo del sistema, agrupadas por capa funcional.	8
2.2. Caracterización resumida del conjunto de grabaciones de video utilizado en el desarrollo y la verificación del sistema.	9
2.3. Modelos preentrenados de terceros incorporados al sistema.	11
2.4. Servicios gestionados de GCP seleccionados para la prueba de concepto.	12

Dedicado a... [OPCIONAL]

Capítulo 1

Introducción general

En este capítulo se contextualiza la enfermedad de Parkinson, la manera habitual de evaluar sus signos motores y se expone la motivación para incorporar análisis automático de video. También se sintetiza el estado del arte en visión por computadora aplicada a trastornos del movimiento y se enuncian los objetivos del trabajo.

1.1. Introducción general al tema

La enfermedad de Parkinson es un trastorno neurodegenerativo frecuente que compromete principalmente el sistema motor. Entre sus manifestaciones se cuentan la bradicinesia, la rigidez, el temblor en reposo y alteraciones de la marcha y el equilibrio. La gravedad y la evolución de estos signos condicionan la calidad de vida del paciente y las decisiones terapéuticas del profesional de la salud.

En la práctica neurológica se emplean escalas validadas que formalizan la exploración motora. Una referencia ampliamente adoptada es la parte motora de la escala unificada de la Sociedad Internacional de Parkinson y trastornos del movimiento (MDS por sus siglas en inglés), conocida en la literatura como MDS-UPDRS parte III, que guía la observación de tareas como movimientos de dedos, postura y marcha [1]. Esas puntuaciones son útiles para comparar pacientes y momentos en el tiempo, pero dependen de la pericia del examinador y del contexto de la consulta, factores que introducen variabilidad entre observadores y entre sesiones.

En los últimos años creció el interés por registrar la exploración en video, ya sea en forma presencial o remota, y por apoyar la lectura clínica con algoritmos que midan trayectorias, ritmo y amplitud a partir de las imágenes [1, 2]. Esa línea de trabajo apunta a obtener descriptores cuantitativos reproducibles a partir de grabaciones convencionales, sin reemplazar al médico pero ofreciéndole un respaldo objetivo para el seguimiento.

Para situar visualmente el tipo de tareas que los profesionales solicitan durante la exploración motora, la figura 1.1 muestra un ejemplo de finger tapping: toques rítmicos entre el pulgar y el índice, frecuentemente incluidos en protocolos como la parte motora de la escala MDS-UPDRS.

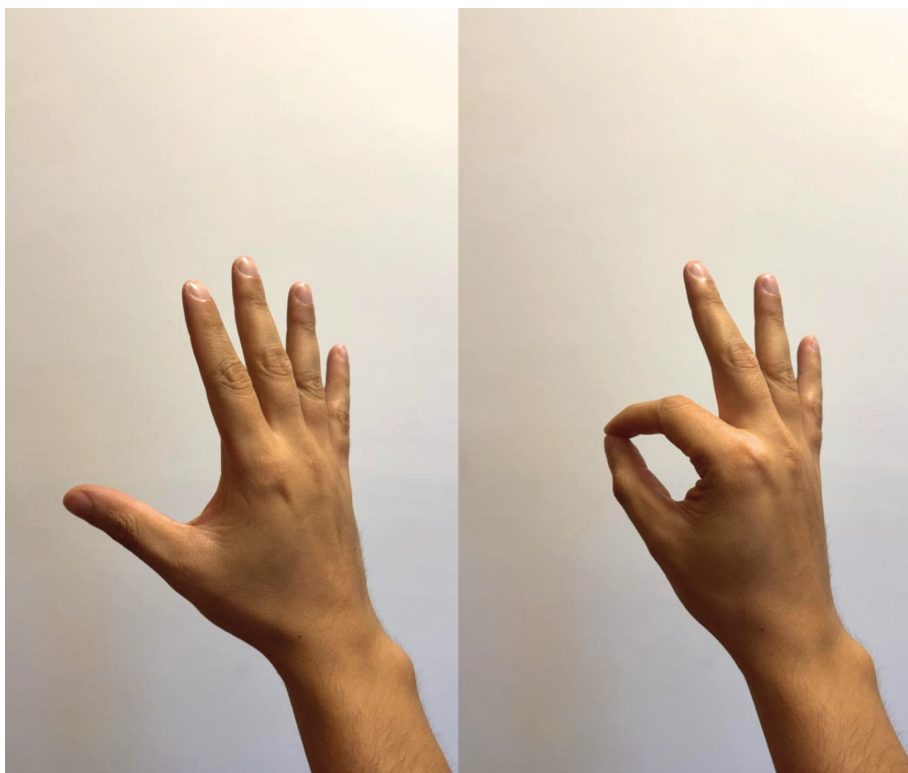


FIGURA 1.1. Ejercicio de finger tapping utilizado en la evaluación clínica de movimientos finos de la mano.

En la imagen se observa la mano en la postura típica de la prueba, que permite al examinador valorar amplitud, regularidad y fatiga del movimiento. Esa misma secuencia puede registrarse en video para analizarla de forma cuantitativa con herramientas de visión por computadora, como se desarrolla en los capítulos posteriores.

1.2. Contexto y motivación

La motivación del trabajo fue construir una herramienta que, a partir de videos de ejercicios motores, estimara la pose corporal del paciente y derivara métricas asociadas a oscilaciones y regularidad del movimiento, de modo de reducir la dependencia exclusiva de la impresión visual humana para apreciar cambios leves o graduales.

La inteligencia artificial y la visión por computadora resultaron adecuadas porque permiten procesar cada fotograma de manera uniforme, conservar el detalle temporal entre cuadros consecutivos y transformar coordenadas articulares en magnitudes interpretables por el personal de salud. Se priorizó un enfoque basado en modelos preentrenados de estimación de pose, alineado con el alcance acordado para el proyecto, que explícitamente no incluyó entrenar redes neuronales desde cero en un conjunto propio de gran escala.

El trabajo culminó con un producto operativo que puede ejecutarse en entorno local y mediante contenedores, y además se desplegó en la nube sobre Google Cloud Platform (GCP), de modo que el acceso al sistema no se limitó a una única máquina de desarrollo. La arquitectura y el procedimiento de despliegue quedaron documentados para facilitar extensiones posteriores, como validación

prospectiva frente a lecturas de referencia o adaptaciones institucionales sujetas a marcos éticos y regulatorios propios de cada jurisdicción.

1.3. Estado del arte

La revisión sistemática de Pecoraro, Marsili, Espay, Bologna y di Biase sobre tecnologías de visión por computadora en trastornos del movimiento resume decenas de estudios entre 1984 y 2024 [2]. En ese conjunto de investigaciones predominan trabajos sobre la enfermedad de Parkinson y tareas de marcha. Para la estimación de pose sin marcadores físicos aparecen con frecuencia implementaciones basadas en OpenPose y, en menor medida, otras soluciones mediante modelos propios o integraciones desarrolladas para cada estudio.

Los autores destacan que el análisis automático de video alcanzó en muchos informes exactitudes diagnósticas superiores al 80 % frente a etiquetas clínicas, con buena concordancia frente a acelerometría en temblor y distonía, aunque advierten heterogeneidad en condiciones de grabación y en el software empleado, lo que dificulta comparar resultados entre sitios.

En paralelo, el trabajo de Sibley, Girges, Hoque y Foltynie analiza retos metodológicos del análisis de video para cuantificar la severidad motora en Parkinson y el papel de la inteligencia artificial y el aprendizaje automático en ese escenario [1]. Concluyen que la evaluación por video es accesible y prometedora, pero que hacen falta muestras amplias y diversas y estándares de desempeño alineados con la práctica clínica antes de generalizar su uso.

Para la estimación de pose en tiempo casi real sobre hardware común, Google publicó el marco MediaPipe, que permite encadenar módulos de visión reutilizables y ajustar el consumo de recursos según el equipo disponible [3]. Dentro de ese ecosistema, el modelo de pose corporal ofrece coordenadas normalizadas de puntos anatómicos en dos y tres dimensiones y se utiliza en prototipos de salud y deporte por su equilibrio entre latencia y precisión aceptable para seguimiento de extremidades.

El prototipo documentado en este trabajo se inscribe dentro del enfoque de visión markerless basado en MediaPipe Pose, orientado al cálculo de métricas de oscilación y al análisis de tareas motoras finas a partir de series de keypoints, en línea con la tendencia observada en la literatura y en las revisiones citadas.

En la tabla 1.1 se contrastan enfoques representativos del estado del arte en cuanto a insumos, salida típica y forma habitual de evaluación reportada en la literatura de revisión. No constituye un inventario exhaustivo de productos comerciales sino una guía conceptual para situar el trabajo desarrollado.

TABLA 1.1. Enfoques frecuentes para cuantificar el movimiento en Parkinson y contextos afines.

Enfoque	Datos de entrada	Salida típica	Evaluación habitual en la literatura
Escalas clínicas (MDS-UPDRS III y otras).	Observación en vivo o video asistido por humano.	Puntuaciones ordinales por ítem.	Consistencia inter-examinador, validez frente a estándares clínicos.
Sensores inerciales.	Aceleración o velocidad angular en segmentos corporales.	Series temporales, estimadores de frecuencia y amplitud.	Concordancia con video o con escala clínica.
OpenPose u otras redes de pose markerless.	Video monocular o multivista.	Coordenadas 2D o 3D de articulaciones.	Exactitud de clasificación o correlación con medidas de referencia [2].
MediaPipe Pose.	Video; modelos pre-entrenados integrados al marco.	Landmarks normalizados 2D/3D y continuidad temporal en un grafo de procesamiento integrado [3].	Idoneidad para tiempo casi real y bajo consumo en dispositivos habituales, frente a estimadores más pesados [3].

1.4. Objetivos del trabajo

Con el contexto clínico, la motivación del análisis automático de video y el panorama de la literatura ya expuestos, a continuación se enuncian el objetivo general y los objetivos específicos que guiaron el desarrollo del sistema.

1.4.1. Objetivo general

Diseñar e implementar un sistema de análisis digital de oscilaciones y desempeño motor en videos de ejercicios clínicos asociados a la enfermedad de Parkinson, apoyado en estimación automática de pose y en técnicas de procesamiento de señales e inteligencia artificial, que entregue métricas cuantitativas y visualizaciones comprensibles para apoyar el seguimiento objetivo del movimiento.

1.4.2. Objetivos específicos

1. Implementar un flujo de ingestión y preprocesamiento de video que alimente de manera estable el estimador de pose y módulos posteriores.
2. Integrar un modelo preentrenado de estimación de pose corporal y un seguimiento temporal de puntos relevantes para los ejercicios considerados.
3. Calcular métricas de amplitud, frecuencia, regularidad y estabilidad postural, u otras derivadas del mismo principio de análisis de series articulares, y agregarlas por sesión para su revisión estadística.
4. Exponer los resultados mediante una interfaz de servicios y una aplicación web que permitan cargar sesiones, inspeccionar gráficos y exportar reportes estructurados.

5. Verificar el comportamiento del software mediante pruebas unitarias y de integración sobre componentes críticos del flujo de procesamiento.

Los criterios de cumplimiento asociados a estos objetivos fueron la ejecución exitosa del flujo completo sobre videos de prueba disponibles para el proyecto, la coherencia de las métricas frente a escenarios controlados o simulados descritos en el capítulo de ensayos, y el paso de la batería de pruebas automatizadas definida en el repositorio del sistema.

Capítulo 2

Introducción específica

En este capítulo se describen las herramientas, bibliotecas, modelos preentrenados y conjuntos de datos provistos por terceros que sostuvieron el desarrollo del sistema. También se detallan el origen y las características de los videos utilizados y la infraestructura de ejecución contemplada para los ensayos y para el despliegue del prototipo.

2.1. Frameworks y bibliotecas

El sistema se construyó sobre tecnologías de código abierto. El núcleo del trabajo es un flujo de procesamiento de video y señales escrito en `Python 3.12`, sobre el que se montan un servicio HTTP, una capa de persistencia y una interfaz web. Esta separación permitió que la misma lógica de procesamiento se pudiera invocar desde una línea de comandos, desde un servicio HTTP o desde un contenedor sin alterar el núcleo del cálculo.

Para la lectura y manipulación de imagen se utilizó `OpenCV` [4], que cubrió la apertura de los archivos de video, la conversión entre espacios de color y las operaciones de redimensionado y muestreo cuadro a cuadro. La extracción del canal de audio asociado a las grabaciones se realizó con `moviepy`, y el análisis posterior de ese canal, en particular para la detección de los pulsos del metrónomo que estructuran los ejercicios, se apoyó en `librosa` [5], una biblioteca especializada en el procesamiento de señales de audio musicales y rítmicas.

El cálculo de las métricas se sostuvo en el conjunto de bibliotecas científicas habituales del ecosistema `Python`, integrado por `NumPy`, `SciPy` y `pandas` [6]. `NumPy` proveyó las operaciones vectoriales sobre las series temporales de los puntos detectados, `SciPy` aportó los estimadores espectrales y los algoritmos de detección de picos utilizados para el cálculo de frecuencia y de regularidad, y `pandas` se utilizó para organizar los datos por cuadro en estructuras tabulares que luego alimentan los pasos de agregación y exportación.

La capa de servicios se desarrolló con `FastAPI` [7], un framework para servicios HTTP en `Python` con soporte nativo para validación de datos a través de `Pydantic`, generación automática de documentación interactiva y eventos enviados desde el servidor (server-sent events) que se utilizaron para informar el progreso de cada análisis en tiempo casi real. La persistencia de pacientes, sesiones y resultados quedó a cargo de `PostgreSQL` [8], una base de datos relacional de uso amplio en aplicaciones de producción, accedida desde la aplicación mediante el object-relational mapper `SQLAlchemy` y administrada mediante `Alembic` para el versionado del esquema.

La interfaz de usuario se construyó como una aplicación web de página única en `TypeScript`, con un conjunto estándar de bibliotecas para componentes visuales, ruteo y gráficos interactivos. El detalle de esa capa se desarrolla en el capítulo de diseño e implementación, donde resulta más pertinente describir la arquitectura del frontend.

En el plano de operación y empaquetado se adoptaron `Docker` [9] y `Docker Compose` para orquestar la base de datos, el servicio HTTP y un servidor web de archivos estáticos en una topología única reproducible en el entorno local y en el destino de despliegue. Las dependencias de `Python` se gestionaron con `uv`, mientras que la calidad del código se verificó mediante análisis estático y un conjunto de pruebas automatizadas con `pytest`.

En la tabla 2.1 se listan las bibliotecas y herramientas más relevantes para el procesamiento de datos y para el funcionamiento del sistema, agrupadas por su rol y con la motivación principal de su elección.

TABLA 2.1. Bibliotecas y herramientas de terceros utilizadas en el desarrollo del sistema, agrupadas por capa funcional.

Capa	Herramientas	Motivo de elección
Visión y video.	MediaPipe, OpenCV.	Modelos preentrenados de pose listos para uso y manipulación robusta de archivos de video.
Audio.	librosa, moviepy.	Extracción del canal de audio y detección de pulsos rítmicos para los ejercicios con metrónomo.
Númerica y métricas.	NumPy, SciPy, pandas.	Operaciones vectoriales, estimadores espectrales y manipulación tabular ampliamente probadas.
Servicios HTTP.	FastAPI, Pydantic.	Validación de datos integrada y soporte nativo para flujos de progreso por SSE.
Persistencia.	PostgreSQL, SQLAlchemy, Alembic.	Base relacional madura, ORM y versionado declarativo del esquema.
Modelo de lenguaje.	OpenAI Python SDK.	Acceso a un modelo de lenguaje generativo para resúmenes interpretativos de las sesiones.
Operación.	Docker, Docker Compose, uv.	Despliegue equivalente en local y nube y dependencias reproducibles.

La elección de tecnologías favoreció componentes de uso difundido y bien documentados, lo que se alineó con los supuestos del proyecto sobre disponibilidad de bibliotecas de código abierto y contribuyó a mitigar el riesgo identificado en la planificación inicial respecto a posibles incompatibilidades en la integración de modelos de visión por computadora.

2.2. Datasets y datos

El trabajo no contempló el entrenamiento de modelos desde cero, por lo que las necesidades de datos se concentraron en disponer de una cantidad de video suficiente para verificar la integración del flujo de procesamiento, evaluar la coherencia de las métricas en escenarios reproducibles y exponer la herramienta a movimientos representativos de los ejercicios contemplados.

No se incorporaron conjuntos de datos públicos. La conformación de un conjunto de grabaciones de pacientes con enfermedad de Parkinson exige un marco de consentimiento informado, anonimización y resguardo bajo regulaciones de protección de datos de salud, condiciones que se inscriben en una etapa posterior a la de un prototipo de investigación. Esa restricción se reflejó desde la planificación inicial, que excluyó explícitamente del alcance la validación clínica formal y la integración con sistemas hospitalarios y dejó la evaluación funcional con especialistas como actividad opcional. En el desarrollo se realizó además un análisis preliminar de los principales puntos de mejora y de criticidad para que el sistema pueda alinearse con normativas vigentes de protección de datos sanitarios, como la *Health Insurance Portability and Accountability Act* (HIPAA) en su jurisdicción de origen. El resultado de ese análisis y las propuestas asociadas se desarrollan en los capítulos posteriores como trabajo futuro y propuestas de mejora.

Para sostener el avance del desarrollo dentro del alcance descripto, se trabajó con dos fuentes complementarias de video: grabaciones propias de personas sanas que reprodujeron los ejercicios estandarizados y un conjunto reducido de grabaciones de pacientes obtenidas en condiciones controladas y bajo acuerdo de uso interno, que se mantuvieron por fuera del repositorio de código.

Esta estrategia resultó adecuada para los objetivos del trabajo porque el sistema se apoya en modelos preentrenados de estimación de pose, cuya capacidad de detectar puntos anatómicos no depende del diagnóstico de la persona registrada. Las grabaciones de personas sanas resultaron suficientes para verificar la robustez del seguimiento de keypoints bajo distintos encuadres y condiciones de iluminación, mientras que las grabaciones de pacientes permitieron observar el comportamiento del sistema en presencia de los patrones motores característicos. La ampliación posterior del conjunto con material clínico anotado se identificó como una línea natural de continuidad y se desarrolla en el capítulo de conclusiones.

Las grabaciones se obtuvieron con la cámara posterior de un teléfono iPhone 15 Pro a 30 cuadros por segundo, en disposición fija sobre una superficie plana y con la región de interés (mano dominante o tronco superior, según el ejercicio) centrada en el cuadro. Durante los primeros segundos de cada grabación se ubicó en el campo visual un tablero de calibración ChArUco [10], que el sistema utiliza para estimar la conversión píxeles–milímetros sin requerir intervención manual del operador. En todas las fuentes del conjunto se registraron los mismos ejercicios: finger tapping, temblor de mano y oscilación de brazos. En la tabla 2.2 se resumen las características de cada fuente de video y las condiciones de captura asociadas.

TABLA 2.2. Caracterización resumida del conjunto de grabaciones de video utilizado en el desarrollo y la verificación del sistema.

Fuente	Notas de captura
Grabaciones propias de personas sanas.	Cámara fija, iluminación ambiente, tablero de calibración al inicio.
Grabaciones reducidas de pacientes.	Obtenidas bajo acuerdo de uso interno, almacenadas por fuera del repositorio.
Material de muestra anonimizado.	Incluido en el repositorio para pruebas de integración continua y para la incorporación de nuevos colaboradores.

El conjunto se administró bajo una clasificación de tres niveles definida en el repositorio del sistema: material de muestra anonimizado, grabaciones reales de pacientes y artefactos de salida del análisis, con políticas explícitas de almacenamiento y de exclusión del control de versiones para los dos últimos. Esa política se documentó internamente para evitar la difusión accidental de datos sensibles y se alinea con los requerimientos éticos enunciados en la planificación inicial.

2.3. Modelos preentrenados

Una de las decisiones de alcance establecidas al inicio del trabajo fue no entrenar redes neuronales desde cero, sino apoyarse en modelos preentrenados de uso difundido. Esa decisión se fundamentó en el equilibrio entre el costo de obtener un conjunto etiquetado de gran tamaño y la disponibilidad de soluciones maduras para la estimación de pose de mano y de cuerpo completo, ya validadas en aplicaciones de salud, deporte y realidad aumentada.

Para la estimación de pose se utilizaron dos modelos del marco MediaPipe [3], distribuidos a través de la API de tareas (Tasks API) en su versión 0.10. El modelo `HandLandmarker` [11] entrega 21 puntos por mano detectada, organizados en falanges y articulaciones, y se utilizó en todos los ejercicios centrados en la mano. El modelo `PoseLandmarker` en su variante lite [12] entrega 33 puntos del cuerpo y se reservó para los ejercicios que requerían el contexto del torso y los brazos. Ambos modelos se descargaron desde el repositorio público de MediaPipe en formato float16 y se almacenaron localmente en una caché versionada de modelos.

En la figura 2.1 se muestra un ejemplo de los 21 puntos de referencia entregados por el modelo `HandLandmarker`, ubicados sobre la imagen de una mano en la postura típica del ejercicio de finger tapping.

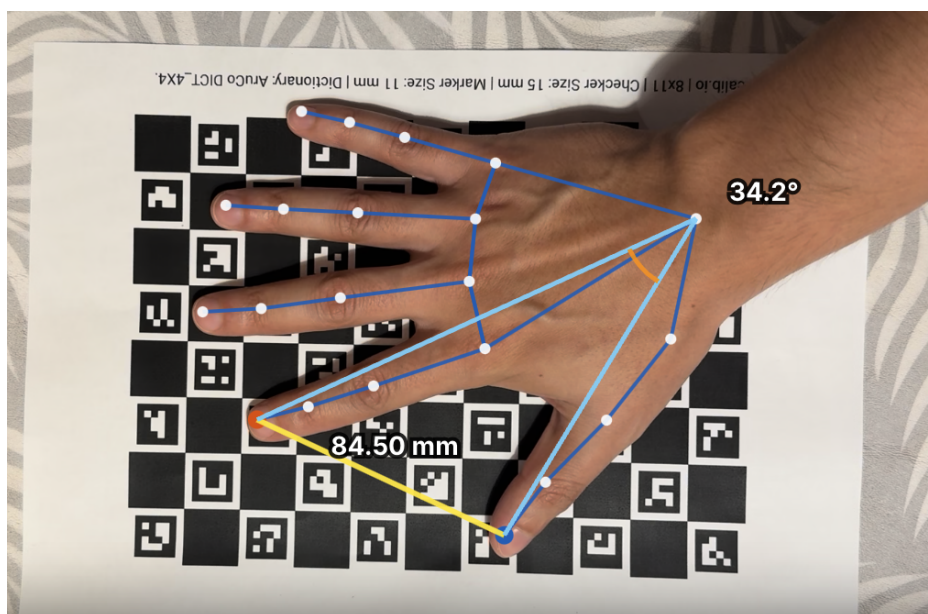


FIGURA 2.1. Ejemplo ilustrativo de los puntos de referencia entregados por el modelo `HandLandmarker` de MediaPipe sobre la imagen de una mano.

En la imagen se observa la disposición regular de los puntos a lo largo de las falanges y las articulaciones, que permite reconstruir la geometría de la mano cuadro a cuadro y, a partir de ella, derivar las distancias y los ángulos sobre los que se calculan las métricas del sistema.

Para la generación automática de resúmenes interpretativos sobre los resultados de cada sesión y sobre la evolución longitudinal de un paciente se incorporó un modelo de lenguaje de gran tamaño (*large language model*, LLM), accedido a través de la API de OpenAI [13].

El modelo elegido fue GPT-5.4-mini, una variante orientada a tareas conversacionales con costo y latencia acotados. El componente del sistema responsable de esta funcionalidad envía únicamente las métricas cuantitativas y los metadatos no identificatorios de la sesión, junto con instrucciones en formato de prompt estructurado. Luego, el sistema recibe un texto en lenguaje natural que el profesional de salud puede inspeccionar como apoyo a la lectura del reporte. El modelo nunca recibe identificadores del paciente, en línea con la política de manejo de datos del proyecto.

En la tabla 2.3 se listan los modelos preentrenados utilizados, indicando su origen, la tarea sobre la que operan y las características relevantes para el sistema.

TABLA 2.3. Modelos preentrenados de terceros incorporados al sistema.

Modelo	Tarea	Origen y datos de pre-entrenamiento	Uso en el sistema
HandLandmarker (MediaPipe 0.10).	Detección de 21 puntos por mano.	Imágenes anotadas de manos en distintas poses, entornos y condiciones de iluminación [11].	Ejercicios de finger tapping y temblor de mano.
PoseLandmarker Lite (MediaPipe 0.10).	Detección de 33 puntos corporales.	Imágenes y video con anotaciones de pose corporal completa [12].	Ejercicios con con- texto de tronco y brazos.
GPT-5.4-mini (OpenAI).	Modelo de lenguaje generativo.	Conjunto general empleado por OpenAI para sus modelos de propósito general [13].	Resúmenes interpretativos por sesión y por evolución a partir de las métricas calculadas.

La integración de los modelos respetó los supuestos del proyecto sobre acceso a soluciones de código abierto y la decisión explícita de no entrenar desde cero. De ese modo se concentró el esfuerzo en el flujo de procesamiento, en la derivación de métricas y en la presentación clínica de los resultados, en lugar de en la construcción de un dataset de entrenamiento y de un protocolo de ajuste de redes neuronales. Otro aporte significativo del trabajo fue la operación e integración de los servicios necesarios para conformar una solución completa, que articula la infraestructura de ejecución, la interfaz de usuario, el servicio de aplicación y la persistencia en una arquitectura coherente y reproducible, descrita en el capítulo de diseño e implementación.

2.4. Infraestructura

Además de las bibliotecas de aplicación, el proyecto contempló tecnologías de infraestructura de terceros para empaquetar el sistema, ejecutarlo en entorno local y desplegarlo en la nube. En esta sección se caracterizan esas plataformas. La topología adoptada y el procedimiento de despliegue se desarrollan en el capítulo de diseño e implementación.

`Docker` [9] es una plataforma de contenedores que empaqueta una aplicación y sus dependencias en una imagen inmutable, ejecutable de forma aislada en sistemas operativos compatibles. `Docker Compose` permite declarar varios servicios interconectados, sus redes y volúmenes persistentes, y levantar la pila completa de manera coordinada. Es un patrón habitual para reproducir entornos de desarrollo sin instalaciones manuales.

Los modelos de estimación de pose son demandantes de memoria RAM pero, en el uso típico de `MediaPipe`, no requieren unidad de procesamiento gráfico dedicada. Debido a eso, la infraestructura prevista se orientó a cómputo en CPU con memoria ampliada, tanto en estaciones locales como en instancias gestionadas en la nube.

Para el despliegue en la nube se seleccionó *Google Cloud Platform* (GCP), ecosistema que admite ejecutar las mismas imágenes de contenedor definidas para el entorno local. En la tabla 2.4 se resumen los servicios gestionados seleccionados para la prueba de concepto de bajo costo en una única región.

TABLA 2.4. Servicios gestionados de GCP seleccionados para la prueba de concepto.

Servicio	Descripción y rol
Cloud Run	Contenedores HTTP serverless con escalado automático, memoria y tiempos de respuesta configurables para cargas largas.
Cloud SQL	PostgreSQL administrado, el Auth Proxy habilita conexión segura sin credenciales embebidas.
Cloud Storage	Objetos duraderos (videos, resultados, modelos) en buckets con prefijos lógicos.
Secret Manager	Secretos versionados inyectados en ejecución.
Cloud Build	Construcción de imágenes y despliegue automatizado hacia Cloud Run.

En conjunto, las herramientas para el despliegue local (`Docker`, `Docker Compose`, `nginx`) y los servicios de GCP (`Cloud Run`, `Cloud SQL`, `Cloud Storage`, `Secret Manager`, `Cloud Build`) cubren cómputo, persistencia, archivos y credenciales de forma portable y gestionada. Configuraciones de mayor exigencia operativa o normativa quedaron fuera del alcance de este trabajo.

Capítulo 3

Diseño e implementación

Todos los capítulos deben comenzar con un breve párrafo introductorio que indique cuál es el contenido que se encontrará al leerlo. La redacción sobre el contenido de la memoria debe hacerse en presente y todo lo referido al proyecto en pasado, siempre de modo impersonal.

3.1. Análisis del software

La idea de esta sección es resaltar los problemas encontrados, los criterios utilizados y la justificación de las decisiones que se hayan tomado.

Se puede agregar código o pseudocódigo dentro de un entorno `lstlisting` con el siguiente código:

```
\begin{lstlisting}[caption= "un epígrafe descriptivo"]
  las líneas de código irían aquí...
\end{lstlisting}
```

A modo de ejemplo, se muestra el fragmento de código [3.1](#):

```
1 #define MAX_SENSOR_NUMBER 3
2 #define MAX_ALARM_NUMBER 6
3 #define MAX_ACTUATOR_NUMBER 6
4
5 uint32_t sensorValue[MAX_SENSOR_NUMBER];
6 FunctionalState alarmControl[MAX_ALARM_NUMBER]; //ENABLE or DISABLE
7 state_t alarmState[MAX_ALARM_NUMBER]; //ON or OFF
8 state_t actuatorState[MAX_ACTUATOR_NUMBER]; //ON or OFF
9
10 void vControl() {
11
12     initGlobalVariables();
13
14     period = 500 ms;
15
16     while(1) {
17
18         ticks = xTaskGetTickCount();
19
20         updateSensors();
21
22         updateAlarms();
23
24         controlActuators();
25
26         vTaskDelayUntil(&ticks, period);
27     }
```

28 }

CÓDIGO 3.1. Pseudocódigo del lazo principal de control.

Capítulo 4

Ensayos y resultados

Todos los capítulos deben comenzar con un breve párrafo introductorio que indique cuál es el contenido que se encontrará al leerlo. La redacción sobre el contenido de la memoria debe hacerse en presente y todo lo referido al proyecto en pasado, siempre de modo impersonal.

4.1. Pruebas funcionales del hardware

La idea de esta sección es explicar cómo se hicieron los ensayos, qué resultados se obtuvieron y analizarlos.

Capítulo 5

Conclusiones

Todos los capítulos deben comenzar con un breve párrafo introductorio que indique cuál es el contenido que se encontrará al leerlo. La redacción sobre el contenido de la memoria debe hacerse en presente y todo lo referido al proyecto en pasado, siempre de modo impersonal.

5.1. Conclusiones generales

La idea de esta sección es resaltar cuáles son los principales aportes del trabajo realizado y cómo se podría continuar. Debe ser especialmente breve y concisa. Es buena idea usar un listado para enumerar los logros obtenidos.

En esta sección no se deben incluir ni tablas ni gráficos.

Algunas preguntas que pueden servir para completar este capítulo:

- ¿Cuál es el grado de cumplimiento de los requerimientos?
- ¿Cuán fielmente se pudo seguir la planificación original (cronograma incluido)?
- ¿Se manifestó algunos de los riesgos identificados en la planificación? ¿Fue efectivo el plan de mitigación? ¿Se debió aplicar alguna otra acción no contemplada previamente?
- Si se debieron hacer modificaciones a lo planificado ¿Cuáles fueron las causas y los efectos?
- ¿Qué técnicas resultaron útiles para el desarrollo del proyecto y cuáles no tanto?

5.2. Próximos pasos

Acá se indica cómo se podría continuar el trabajo más adelante.

Bibliografía

- [1] Krista G. Sibley, Christine Girges, Ehsan Hoque y Thomas Foltynie. «Video-Based Analyses of Parkinson's Disease Severity: A Brief Review». En: *Journal of Parkinson's Disease* 11.Suppl. 1 (2021), S83-S93. DOI: [10.3233 / JPD-202402](https://doi.org/10.3233/JPD-202402).
- [2] Pasquale Maria Pecoraro, Luca Marsili, Alberto J. Espay, Matteo Bologna y Lazzaro di Biase. «Computer Vision Technologies in Movement Disorders: A Systematic Review». En: *Movement Disorders Clinical Practice* 12.9 (2025), págs. 1229-1243. DOI: [10.1002/mdc3.70123](https://doi.org/10.1002/mdc3.70123).
- [3] Camillo Lugaresi, Jiuqiang Tang, Hadon Nash, Chris McClanahan, Esha Uboweja, Michael Hays, Fan Zhang, Chuo-Ling Chang, Ming Guang Yong, Juhyun Lee, Wan-Teh Chang, Wei Hua, Manfred Georg y Matthias Grundmann. «MediaPipe: A Framework for Building Perception Pipelines». En: *arXiv preprint arXiv:1906.08172* (2019).
- [4] Gary Bradski. «The OpenCV Library». En: *Dr. Dobb's Journal of Software Tools* (2000).
- [5] Brian McFee, Colin Raffel, Dawen Liang, Daniel P. W. Ellis, Matt McVicar, Eric Battenberg y Oriol Nieto. «librosa: Audio and Music Signal Analysis in Python». En: *Proceedings of the 14th Python in Science Conference (SciPy)*. 2015, págs. 18-25. DOI: [10.25080/Majora-7b98e3ed-003](https://doi.org/10.25080/Majora-7b98e3ed-003).
- [6] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C J Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa y Paul van Mulbregt. «SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python». En: *Nature Methods* 17 (2020), págs. 261-272. DOI: [10.1038/s41592-019-0686-2](https://doi.org/10.1038/s41592-019-0686-2).
- [7] Sebastián Ramírez. *FastAPI*. <https://fastapi.tiangolo.com/>. Disponible: 2026-04-15. 2024.
- [8] The PostgreSQL Global Development Group. *PostgreSQL: The World's Most Advanced Open Source Relational Database*. <https://www.postgresql.org/>. Disponible: 2026-04-15. 2024.
- [9] Dirk Merkel. «Docker: Lightweight Linux Containers for Consistent Development and Deployment». En: *Linux Journal* 2014.239 (2014).
- [10] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas y M. J. Marín-Jiménez. «Automatic generation and detection of highly reliable fiducial markers under occlusion». En: *Pattern Recognition* 47.6 (2014), págs. 2280-2292. DOI: [10.1016/j.patcog.2014.01.005](https://doi.org/10.1016/j.patcog.2014.01.005).

- [11] Fan Zhang, Valentin Bazarevsky, Andrey Vakunov, Andrei Tkachenka, George Sung, Chuo-Ling Chang y Matthias Grundmann. *MediaPipe Hands: On-device Real-time Hand Tracking*. arXiv preprint arXiv:2006.10214. 2020. URL: <https://arxiv.org/abs/2006.10214>.
- [12] Valentin Bazarevsky, Ivan Grishchenko, Karthik Raveendran, Tyler Zhu, Fan Zhang y Matthias Grundmann. *BlazePose: On-device Real-time Body Pose tracking*. arXiv preprint arXiv:2006.10204. 2020. URL: <https://arxiv.org/abs/2006.10204>.
- [13] OpenAI. *OpenAI API Reference*. <https://platform.openai.com/docs/api-reference>. Disponible: 2026-04-15. 2026.